
DATA COMPRESSION CHALLENGE - LOSSLESS COMPRESSION

Eduardo Lopes
Nmec 103070
University of Aveiro
eduardolopes@ua.pt

Rodrigo Abreu
Nmec 113626
University of Aveiro
rodrigo.abreu@ua.pt

Hugo Ribeiro
Nmec 113402
University of Aveiro
hugo.ribeiro04@ua.pt

April 13, 2026

ABSTRACT

We present three lossless data compressors designed to explore the trade-off between compression ratio, compression and decompression speed. Each tool pairs a custom modeling stage with a byte-level range coder, and targets a different corner of this trade-off space: *ox-ratio* maximizes compression ratio, *ox-fast* minimizes latency, and *ox-balanced* seeks the best ratio within a tight time budget. All three compressors were implemented in C++ and verified lossless on a diverse benchmark suite spanning text, binary, and genomic data.

1 Introduction

Lossless data compression exploits statistical regularities in a source to produce a shorter representation that can be reconstructed exactly. Any such system can be decomposed into two stages: a *modeling* stage, which estimates the probability distribution of the next symbol given the data seen so far, and a *coding* stage, which maps those estimates to near-optimal bit lengths. The theoretical lower bound on the expected code length is the entropy $H = -\sum_i p_i \log_2 p_i$ bits per symbol. Compression is only possible to the extent that the source deviates from uniformity.

In practice, compressor design involves navigating a three-way trade-off between compression ratio, compression speed, and decompression speed. Different use cases weight these dimensions differently: archival storage favours ratio, interactive pipelines favour speed, and streaming applications may prioritise fast decompression above all else.

This work presents three independent compressors, each targeting a different region of this trade-off space. *ox-ratio* pursues the lowest achievable bits-per-symbol regardless of computational cost. *ox-fast* minimises total processing time while maintaining competitive compression. *ox-balanced* seeks the best compression ratio within a tight time budget. All three tools use a range coder as the coding stage (*ox-fast* and *ox-balanced* at byte granularity, *ox-ratio* at bit granularity to feed its context-mixing ensemble) and were benchmarked against *gzip*, *bzip2*, *lzma*, *xz*, and *zstd* on a 12.5 MB suite of files with varied statistical properties.

The remainder of this paper is organized as follows: Sections 2, 3, and 4 describe each compressor independently. Section 5 presents benchmark results against state-of-the-art tools and Section 6 concludes.

2 Compressor focused on speed

2.1 Modeling Stage

The input is divided into blocks of 256 KiB, and each block is encoded independently. Within a block, the model maintains frequencies for the 256 possible byte values and predicts the next symbol using only the empirical byte distribution observed so far in that block. This is a fully adaptive ORDER-0 model: all frequencies are initialised to 1, forming a uniform prior, and the frequency of each symbol is incremented immediately after encoding it.

To keep cumulative-frequency queries efficient, the model is implemented with a Fenwick tree. This data structure supports both prefix-sum queries and point updates in $\mathcal{O}(\log N)$ time, which is sufficient for adaptive range coding

while keeping per-symbol overhead small. Since the alphabet is fixed at $N = 256$, the model remains compact and cache-friendly throughout execution.

After a block is encoded, the compressor compares the compressed payload size with the original block size. If the range-coded payload is not smaller, the block is emitted verbatim in raw mode instead. This raw-block fallback is particularly important for a speed-oriented compressor, since it avoids wasting both time and space on blocks that are already close to incompressible.

Compression is parallelised at block granularity. Up to eight worker threads process blocks independently, each with its own fresh adaptive model. Because blocks do not share statistical state, there is no cross-thread synchronization in the hot modeling path. The main thread only needs to preserve the original block order when writing the final output stream.

2.2 File Format and Decompression

The output begins with a four-byte magic signature, followed by the configured block size stored as a 32-bit little-endian integer. The remainder of the file is a sequence of independent blocks. Each block stores a one-byte mode field, a 32-bit raw size, a 32-bit payload size, and then the payload itself. Mode 0 denotes a raw block, whose payload is copied directly to the output during decompression; mode 1 denotes a range-coded block.

Decompression mirrors the compressor exactly. For raw blocks, the payload is written directly. For range-coded blocks, the decoder initialises a fresh adaptive ORDER-0 model and reconstructs exactly `raw_size` bytes by querying the same cumulative-frequency intervals in the same order as the encoder. Because both sides start from the same uniform prior and update the model after every symbol, the original block is recovered losslessly.

As in compression, decompression exploits block independence to run in parallel. Block payloads are buffered, assigned to worker threads, and decoded independently. Completed blocks may finish out of order, but they are committed to the final output in their original sequence. This ordered-parallel pipeline gives *ox-fast* a low total runtime without requiring a more complicated container format.

3 Compressor focused on ratio

ox-ratio pursues the lowest achievable bits-per-symbol, accepting arbitrarily high computation time to reach it. The design centers on a context-mixing ensemble that predicts each bit individually, paired with a block transform pipeline that reshapes the data into a form the model can predict with near-certainty. The coding stage uses a binary range coder [1] with $k_{\text{total}} = 4096 = 2^{12}$, so the hot-path division is a right-shift by 12, and carry propagation follows the LZMA scheme, buffering 0xFF bytes until a non-carry byte resolves them in $\mathcal{O}(1)$.

3.1 Modeling Stage

MI pre-filter. The input is divided into 4 MiB blocks. Before applying any transform, the bigram mutual information $MI = 2H(\text{bytes}) - H(\text{bigrams})$ is computed in a single $\mathcal{O}(N)$ pass, where $H(\text{bytes})$ is the ORDER-0 byte entropy and $H(\text{bigrams})$ is the joint entropy of consecutive byte pairs. MI quantifies how strongly adjacent bytes are correlated. When $MI < 0.10$, the bytes are nearly independent: BWT would permute them without grouping any contexts, and MTF would not produce the zero-biased output the CM model relies on. Such blocks are therefore encoded *directly* with the CM model, bypassing both transforms entirely. The model still captures residual higher-order structure through its context maps, typically reaching below the ORDER-0 entropy floor (file A improves from 4.272 to 4.154 bpb this way). Bit 31 of the per-block tag in the file header records which path was taken so the decompressor can skip the inverse transforms accordingly.

BWT and MTF. When MI is sufficient, the Burrows-Wheeler Transform [3] is applied using SA-IS [4] in $\mathcal{O}(N)$ time on the doubled string $T + T + \$$, which yields cyclic-rotation order without an $\mathcal{O}(N^2)$ comparison sort. BWT groups bytes that share the same preceding context into adjacent runs; Move-to-Front then encodes each byte as its rank in a recently-used list, converting those runs into long sequences of zeros and small integers. This is the critical property that makes the CM model effective: the model can assign near-certain probabilities to most bits and pay almost no bits to encode them.

Context Mixing Model. Rather than using a single statistical model, *ox-ratio* maintains an ensemble of seven sub-models that each observe a different amount of history. The stationary model tracks global bit-position statistics; ctx0 conditions only on the current partial byte; ctx1–ctx4 are hash-table context maps of growing order (4K to 4M

entries), each adding one more previous byte to the key; and runCtx (128K entries) conditions on the current run length and the previous byte, capturing patterns such as “after many consecutive zeros, a non-zero byte is more likely.” Every hash-table entry stores a (c_0, c_1) observation count pair, initialised at $(1, 1)$ as a Laplace prior and halved whenever their sum exceeds 1024 to keep estimates responsive to recent data.

Combining seven probability estimates by simple averaging would undercount the evidence: if two *independent* models both predict 90%, the true combined confidence should be $\approx 99\%$, not still 90%. The correct way to aggregate independent evidence is in *log-odds* (logit) space, where probabilities add instead of saturate. Each estimate $\hat{p}_i$ is converted to its log-odds, scaled by a per-model adaptive weight w_i , and the weighted mean is converted back to a probability: $p_{\text{mix}} = \sigma(\sum_i w_i \ln \frac{\hat{p}_i}{k - \hat{p}_i} / \sum_i w_i) \cdot k$. After observing each bit, every weight is nudged up or down by a small gradient step proportional to how accurate that model’s prediction was, then clamped to $[k_w/16, 16k_w]$ so no single model can dominate or be silenced. The *stretch* and *squash* conversions are precomputed into 4096-entry lookup tables to avoid transcendental calls in the inner loop.

SSE calibration. The mixed probability p_{mix} is calibrated by two successive SSE layers that correct for model correlation and systematic bias. Layer 1 maintains adaptive count pairs indexed by $(\text{bitPos}, \lfloor p_{\text{mix}} \times 32/k_{\text{total}} \rfloor)$ (32 probability buckets per bit position). Layer 2 adds conditioning on the high nibble of *prev0* (values 0–15). The final probability blends both: $p_{\text{final}} = (2p_{\text{SSE1}} + p_{\text{SSE2}})/3$.

3.2 File Format and Decompression

The file begins with magic "TA10", original size (8 B), block size (4 B), block count (4 B), and one 32-bit tag per block. Bit 31 of each tag is the direct-mode flag; when clear, bits 30–0 hold the BWT primary index. Decompression reads the header, then reconstructs each block independently: the range decoder queries `ContextMixModel::predict()` bit by bit, mirroring the encoder exactly, and applies MTF inverse followed by BWT inverse via LF-mapping in $\mathcal{O}(N)$ for BWT+MTF blocks [3].

4 Compressor focused on balance

4.1 Coding Stage

The coding stage was developed in two iterations. Initially, the reference arithmetic coder by Project Nayuki [2] was used, a bit-level encoder that renormalises after every bit, resulting in up to $8\times$ more renormalisation steps than necessary. Its frequency table rebuilt the full cumulative sum array after every symbol update, costing $\mathcal{O}(N)$ per operation where N is the alphabet size.

The first improvement replaced the flat frequency table with a Fenwick tree (Binary Indexed Tree). A Fenwick tree supports both prefix-sum queries and point updates in $\mathcal{O}(\log N)$, which is what the arithmetic coder requires to compute the cumulative frequency interval for each symbol. For an alphabet of up to 257 symbols and millions of coded symbols per file, this reduced the dominant bottleneck from linear to logarithmic cost.

The second improvement replaced the bit-level arithmetic coder with a byte-level range coder adapted from Igor Pavlov’s 7-Zip source [1]. Instead of emitting one bit at a time during renormalisation, the encoder buffers output one byte at a time, using a carry-propagation mechanism to handle overflow correctly. This reduces the number of renormalisation steps by approximately $8\times$ and eliminates the bit-level I/O overhead of the original coder.

4.2 Modeling Stage

The entire input is loaded into memory in a single bulk read. A single sequential pass over the data simultaneously computes two quantities: the ORDER-0 byte entropy $H = -\sum_b p(b) \log_2 p(b)$ and the bigram mutual information $\text{MI} = 2H(\text{bytes}) - H(\text{bigrams})$, where $H(\text{bigrams})$ is the entropy of consecutive byte pairs. MI measures how strongly adjacent bytes are correlated: high MI indicates that BWT clustering will be effective; low MI indicates that bytes are nearly independent and BWT provides no benefit.

Based on these two measures, the pipeline selects one of two paths before any heavy computation begins. If $H > 7.5$ bpb the file is near-random (e.g. already compressed or encrypted) and the ORDER-0 path is taken directly. Otherwise, if $\text{MI} < 0.45$ the adjacent bytes carry little mutual information and BWT would produce no useful clustering, so ORDER-0 is again selected. Only when both conditions fail does the pipeline proceed with the full BWT+MTF transform. On the benchmark suite, files A, D, and E route to ORDER-0; files B, C, F, G, and H route to BWT+MTF. This pre-filter ensures that near-incompressible files never pay the cost of the $\mathcal{O}(N)$ BWT transform.

When the ORDER-0 path is selected, a Fenwick-tree frequency table over 256 symbols is initialised with a uniform prior. Each byte is range-coded using the current symbol probabilities, after which its frequency is updated. This fully adaptive model converges toward the empirical byte distribution as encoding proceeds.

When the BWT+MTF path is selected, the ORDER-0 encoder is simultaneously launched as a background thread. For files where BWT produces little clustering, both outputs are compared at the end and the smaller one is written (at zero extra wall-clock cost).

BWT Forward Transform. The Burrows-Wheeler Transform [3] is applied using the SA-IS algorithm [4], which builds the suffix array in $\mathcal{O}(N)$ time and space. To obtain cyclic rotation order from a linear suffix sorter, the input is doubled to a string $T + T + \$$ of length $2N + 1$. Sorting the first N suffixes of this string gives the cyclic rotation order of T . The simpler $T + \$$ construction is incorrect for periodic inputs and was rejected.

Run-Length Encoding. The BWT output is scanned and consecutive identical bytes are collapsed into (symbol, count) pairs. BWT tends to cluster identical bytes into long runs, making this step very effective on structured data.

Chunk Splitting and Parallel MTF. The run list is divided into T equal sub-lists, where T is the hardware thread count. Each chunk is encoded independently in a separate parallel thread. The MTF list and all statistical models are initialised fresh per chunk, which preserves independence for parallel decoding.

Within each chunk, Move-To-Front coding is applied: each symbol’s rank in the MTF list is encoded, then the symbol is moved to the front. BWT clustering concentrates ranks near 0, which the range coder encodes cheaply.

Compact Alphabet. Each chunk records only the $k \leq 256$ distinct byte values that appear in it. If $k < 256$, the alphabet is stored explicitly (k bytes). If $k = 256$, it is omitted and reconstructed by the decompressor as $[0, 1, \dots, 255]$.

Adaptive Rank and Count Models. Ranks are coded with a Fenwick-tree frequency table over $k + 1$ symbols (ranks $0 \dots k - 1$ plus an end-of-chunk marker), updated adaptively after each symbol. Counts are coded with four context-dependent models based on rank: $r = 0$, $r = 1$, $r \in \{2, 3\}$, and $r \geq 4$. Each context maintains its own adaptive frequency table of 9 symbols, where the first 8 represent counts 1–8 directly. Counts above 8 are signalled by the ninth symbol followed by an Elias-gamma suffix: the exponent $b = \lfloor \log_2(\text{count}) \rfloor$ is range-coded adaptively, and the b -bit residual is coded bit by bit.

Output Format. The compressed file begins with a single mode byte: 0x00 for the BWT+MTF path and 0x01 for ORDER-0. In ORDER-0 mode, the header contains the original file size and the bitstream size, followed by the range-coded byte stream. In BWT+MTF mode, the header stores the BWT primary index (the position in the sorted rotation matrix corresponding to the original string, required for inversion) and the number of chunks. Each chunk then records its alphabet size, the explicit alphabet if smaller than 256 symbols, and its range-coded bitstream. All multi-byte integers are stored in little-endian order.

4.3 Decompression

Decompression begins by reading the mode byte to determine which path was used. In ORDER-0 mode, the original file size is read from the header and the range decoder reconstructs each byte by querying the same adaptive Fenwick-tree model, updating it after every decoded symbol to mirror the encoder exactly.

In BWT+MTF mode, the primary index and chunk headers are read sequentially and all chunk bitstreams are buffered in memory. Each chunk is then decoded in a separate parallel thread, mirroring the encoder: the range decoder recovers MTF ranks and run counts using the same adaptive models initialised with the same priors, the move-to-front list is reversed to recover the original BWT symbols, and runs are expanded back into the full BWT sequence. Once all threads complete, their outputs are concatenated in order to reconstruct the complete BWT-transformed sequence. Finally, the BWT inverse is applied using LF-mapping, a single left-to-right pass builds the predecessor array from the frequency counts of each byte, and a second pass follows the chain from the primary index to reconstruct the original file in $\mathcal{O}(N)$ time.

5 Benchmark

All three compressors were evaluated on a suite of eight files (A–H, totalling 12.5 MB) covering a broad range of data types: A is plain ASCII text, B is C source code, C is UTF-8 encoded text, D is pseudo-random binary data, E is a

floating-point numerical array, F is high-entropy binary data, G is 16-bit PCM audio, and H is a compiled x86-64 ELF executable. Each file was compressed and decompressed independently and the results averaged across all eight files (per-file mean). The benchmark was run against `gzip`, `bzip2`, `lzma` (levels 1, 5, and 9), `xz-6`, and `zstd` (levels 1, 3, and 19) on the same machine. Table 1 reports the per-file mean results and Table 2 shows the per-file breakdown for our three compressors.

Table 1: Per-file mean benchmark results (average over files A–H).

Compressor	bits/byte	t_{comp} (s)	t_{decomp} (s)	t_{total} (s)
ox-ratio	4.325	2.045	1.808	3.853
xz-6	4.332	0.429	0.029	0.458
lzma-9	4.348	0.458	0.040	0.498
lzma-5	4.351	0.404	0.040	0.444
ox-balanced	4.378	0.201	0.054	0.255
bzip2	4.390	0.099	0.051	0.150
zstd-19	4.477	0.388	0.000	0.388
lzma-1	4.545	0.154	0.040	0.194
zstd-3	4.687	0.009	0.000	0.009
gzip	4.758	0.065	0.007	0.072
zstd-1	4.802	0.004	0.000	0.004
ox-fast	5.675	0.021	0.016	0.037

Table 2: Compression ratio (bits/byte) per file for our three compressors.

File	ox-ratio	ox-balanced	ox-fast
A	4.154	4.165	4.190
B	1.355	1.395	5.301
C	2.015	2.067	5.225
D	8.002	8.001	8.000
E	6.961	7.053	7.060
F	6.431	6.589	7.008
G	2.270	2.280	3.280
H	3.402	3.494	6.141
Mean	4.325	4.378	5.675

The per-file results reflect the statistical properties of each file rather than compressor quality alone. Files D and E are near-random (file D has byte entropy above 7.9 bpb and file E has very low bigram mutual information) meaning their bytes carry little exploitable structure. No compressor achieves meaningful compression on these files. The results near 8 bpb represent a fundamental information-theoretic limit rather than a weakness of any particular tool. Files B, C, G, and H, by contrast, exhibit strong inter-byte correlations and all compressors benefit significantly, with our tools achieving their best results on these inputs.

ox-ratio achieves the best compression ratio overall at 4.325 bpb, outperforming `xz-6` (4.332) and all `lzma` levels, at the cost of significantly higher runtime. The MI pre-filter contributes notably on near-independent files: file A, where BWT provides no benefit, is encoded at 4.154 bpb, which is below its ORDER-0 entropy floor, by routing directly through the context-mixing model. *ox-ratio*’s high decompression time (1.808 s) reflects that there is no fast inverse path: the context-mixing model must be re-evaluated bit by bit during decoding, making decompression as costly as compression.

ox-balanced performs competitively across most files but falls behind `xz-6` on files H and F. This is expected: LZMA’s dictionary-based matching captures long-range repetitions spanning thousands of bytes, whereas BWT operates on the entire file as a single block and encodes correlations through symbol clustering rather than explicit back-references. For files where long-range structure dominates, dictionary methods retain a systematic advantage.

ox-fast trades compression ratio for low total runtime. Its weaker mean result of 5.675 bpb is expected, since its adaptive ORDER-0 block model is much simpler than the transform-based and higher-context methods used in the other two compressors. This limits the amount of structure it can exploit, especially on highly compressible files, but keeps overhead small and allows efficient parallel block processing.

Table 2 also reveals a convergence point: on file A (plain ASCII text, MI=0.013), all three compressors achieve nearly identical ratios of 4.154, 4.165, and 4.190 bpb respectively. When adjacent bytes carry almost no mutual information, BWT produces no useful clustering and higher-order context models find no patterns to exploit. In this regime the

adaptive ORDER-0 model is essentially optimal, and the three designs converge because the data itself leaves no room for a more sophisticated model to improve upon it.

6 Conclusion

This work presented three lossless compressors targeting different regions of the compression trade-off space. The benchmark results confirm that no single design dominates across all dimensions: *ox-ratio* achieves the lowest bits-per-symbol (4.325 bpb, outperforming xz-6) at the cost of compression time, *ox-fast* minimises latency (0.037 s total round-trip) at the cost of ratio, and *ox-balanced* sits competitively between the two extremes, surpassing bzip2 in ratio while compressing more than twice as fast as xz-6.

Each design is shaped by a distinct central insight. *ox-fast* achieves low latency through block-level parallelism: the input is split into fixed-size independent blocks, each compressed by a worker thread with its own adaptive ORDER-0 model. An ordered parallel processor collects results in submission order without stalling the main thread, and a per-block raw-fallback ensures incompressible blocks are never expanded. *ox-ratio* achieves its compression depth through context mixing: seven sub-models ranging from a stationary bit-position model to ORDER-4 byte-context models and a run-length context are blended bit by bit with online-adapted weights, and two secondary symbol estimation layers further calibrate the mixed prediction against recent outcomes. The result is a model that automatically favours deeper contexts when they are predictive and falls back to shallower ones when the data is sparse. *ox-balanced* exploits the observation that a cheap linear-time pre-filter based on byte entropy and bigram mutual information can reliably predict whether a transform-based pipeline will outperform a direct adaptive coder. Combined with parallel chunk processing and contextual adaptive models, this avoids expensive computation on near-incompressible inputs entirely, and produces a compressor well-suited to metrics that simultaneously weight compression ratio, compression speed, and decompression speed.

Future work could extend each compressor independently. For *ox-balanced*, incorporating a higher-order context model on top of the BWT output would better capture the long-range dependencies that currently give LZMA a systematic advantage on certain file types. For *ox-fast*, augmenting the current adaptive ORDER-0 model with a lightweight ORDER-1 fallback could improve compression ratio on structured files without sacrificing the low-overhead design. For *ox-ratio*, integrating a match model into the context-mixing ensemble (a hash table mapping recent byte contexts to positions in a circular history buffer) would allow the compressor to detect and exploit long-range repetitions, directly addressing the remaining gap with LZMA on files dominated by repeated sequences.

References

- [1] I. Pavlov. 7-Zip source code, CPP/7zip/Compress/RangeCoder.h. Igor Pavlov, 1999–2024. Available: <https://github.com/ethereon/p7zip-hybrid/blob/master/p7zip/CPP/7zip/Compress/RangeCoder.h>
- [2] Project Nayuki. Reference arithmetic coding. MIT License, 2022. Available: <https://www.nayuki.io/page/reference-arithmetic-coding>
- [3] M. Burrows and D. J. Wheeler. A block-sorting lossless data compression algorithm. Technical Report 124, Digital Equipment Corporation, 1994. Available: https://www.cs.jhu.edu/~langmea/resources/burrows_wheeler.pdf
- [4] G. Nong, S. Zhang, and W. H. Chan. Linear suffix array construction by almost pure induced-sorting. In *Proceedings of the 2009 Data Compression Conference (DCC)*, pages 193–202. IEEE, 2009. Available: <https://ieeexplore.ieee.org/document/4976463>